Building a Civic Evidence Architecture

An architecture framework for Toronto Public Transit Service Pulse

From public feeds and grouped community data
to transparent, bounded and useful civic evidence

The architecture begins with a question, not a dashboard.

This article explains the platform's product boundaries, evidence model, technology responsibilities, safeguards and publication approach.



Platform reference article | October 2026

OVERVIEW

Architecture for public meaning, not just system delivery

Toronto Public Transit Service Pulse began with a straightforward idea: public transit data should help people understand how the city works. That goal sounds simple until live vehicle reports, schedules, walking routes, community context, historical records and experimental models begin to meet on the same page. At that point architecture is no longer only a technical concern. It determines what the product can responsibly claim.

A conventional dashboard architecture often starts with the data that is available and asks how to visualize it. This platform takes the opposite route. It starts with a civic question, defines the evidence that can support an answer, preserves the limits of that evidence, and then chooses the lightest technical design capable of delivering it.

The result is best described as a civic evidence architecture: a controlled path from public records to public meaning.

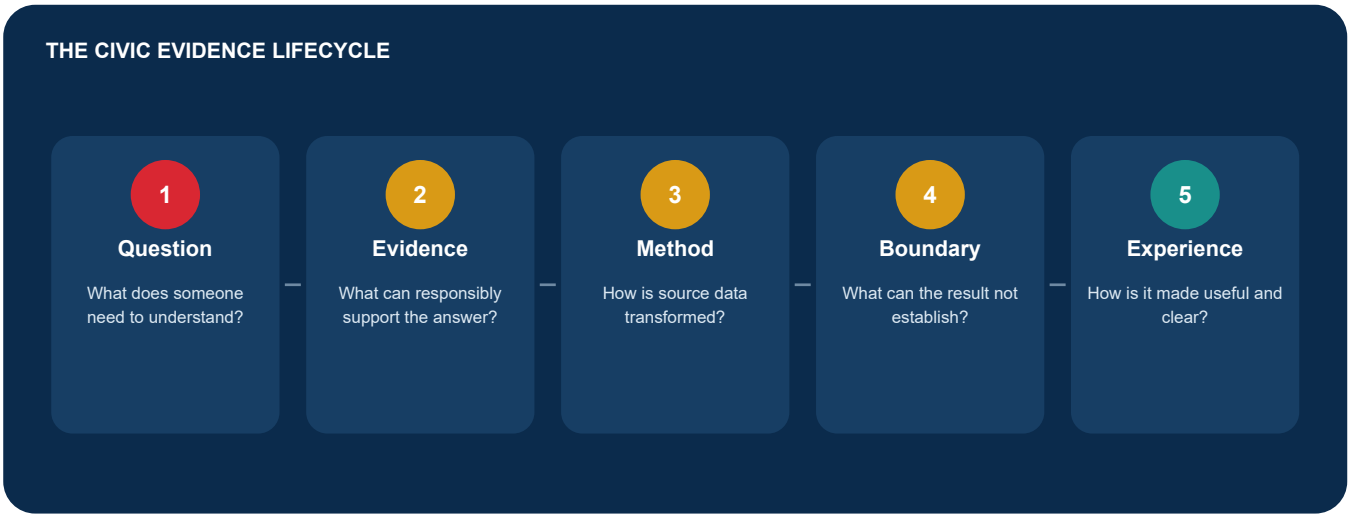


Figure 1. The platform's design sequence keeps interpretation and user experience connected to the original civic question.

The central design objective

Produce the most credible and useful public evidence possible for each unit of compute, storage, network traffic and maintenance effort. This is a value test, not simply a cost-cutting exercise.

01 / PLATFORM CONTEXT

One platform, four deliberately different spaces

The platform has grown beyond a public website, but its parts should not blur together. Each surface has a different audience, risk level and publication rule.

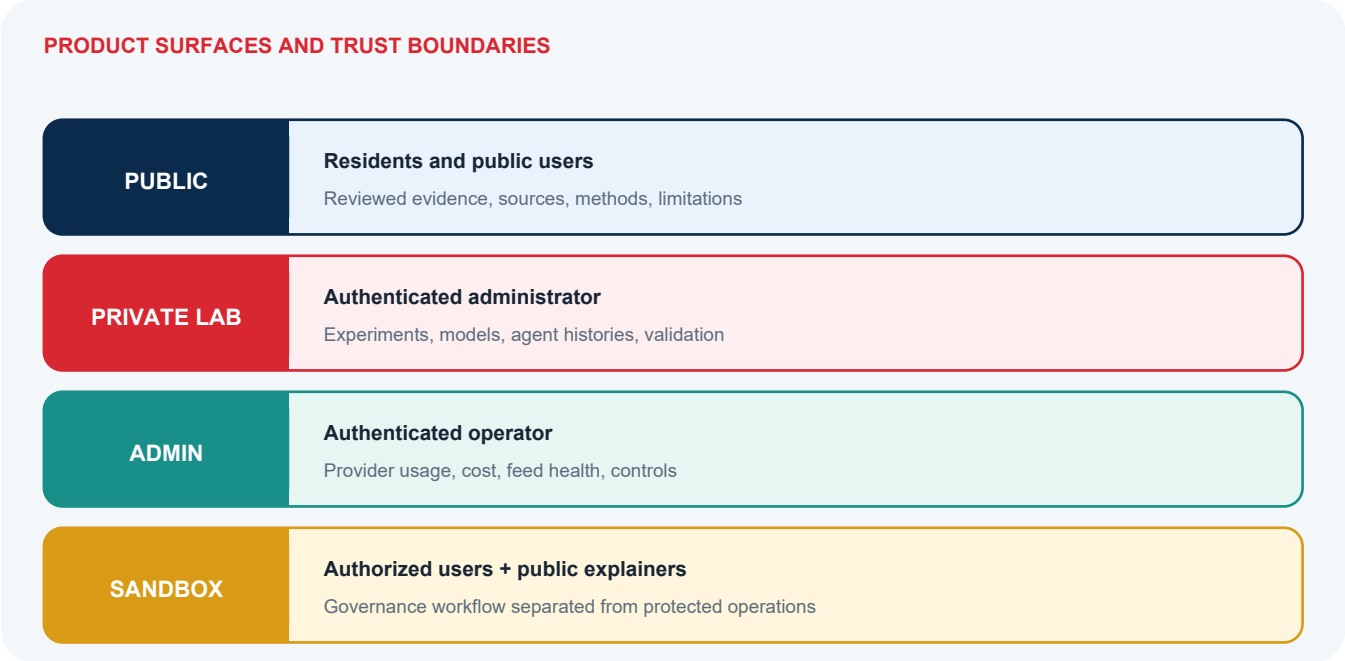


Figure 2. A shared platform does not mean shared access. Product surfaces carry different trust and publication boundaries.

Surface	Audience	Purpose	Rule
Public transit product	Residents, advocates, researchers, journalists and public institutions	Explain service, access, routes, journeys and change over time	Reviewed evidence only
Private Transit Lab	Administrator and approved researchers	Develop and validate models, agents and civic reports	Authenticated and no-store
Administration	Platform operator	Monitor cost, provider usage, data health and controls	Operational access only
DatalyIQ Sandbox	Authorized users and selected public explainers	Test governance, architecture, quality and cross-product workflows	Public explanation separated from protected operations

The domain boundary reinforces the product boundary. transit.datalyiq.com is the public civic transit product. sandbox.datalyiq.com is a cross-product testing environment that can connect transit, Community Atlas and future products without presenting sandbox output as public evidence.

02 / LOGICAL DESIGN

A controlled flow from source to experience

The platform is not a collection of unrelated pages. It is a pipeline in which acquisition, validation, classification, storage and interpretation each have a distinct role.

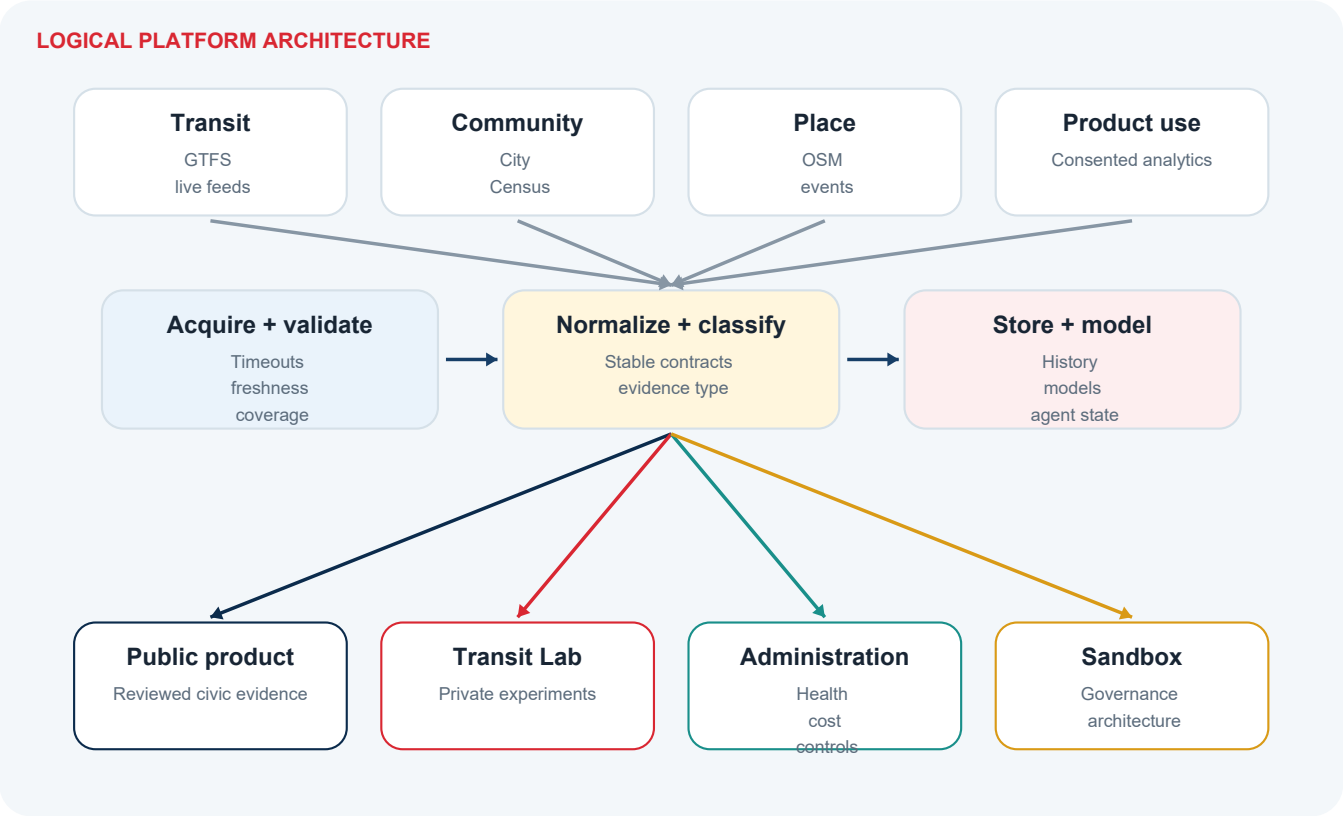


Figure 3. Public and approved sources move through validation and evidence layers before they reach a product experience.

The architecture preserves a crucial separation: upstream sources report facts in their own formats, while the platform publishes stable, source-aware contracts. Pages do not need to understand every detail of GTFS-Realtime, Census tables or OpenStreetMap queries. They consume normalized evidence with timestamps, quality states, coverage and limitations attached.

This separation also makes the product more resilient. If a feed changes or becomes temporarily unavailable, the source adapter and quality layer can respond without forcing every interface to invent its own failure behavior.

Why this matters civically

A missing vehicle report, a true zero, an unavailable occupancy category and a stale feed are not the same condition. The architecture must retain those differences so the product does not turn uncertainty into a false conclusion.

03 / EVIDENCE ARCHITECTURE

Five evidence classes prevent one number from pretending to be another

Transit products often place live positions, schedules, estimates and demographic context beside one another. The danger is not the combination itself. The danger is allowing proximity on a screen to imply that every value has the same status.

Class	Meaning	Example	Interpretation boundary
Observed	A source reported a condition or event	Vehicle visible in a recent TTC feed	Does not prove complete service or passenger experience
Scheduled	A published plan describes intended service	Trip in static TTC GTFS	Does not prove the trip operated
Estimated	A transparent calculation approximates a condition	Grouped population within a walking screen	Depends on assumptions and source coverage
Modeled	A versioned method explores or forecasts a result	Disruption or journey scenario	Not an observed outcome or TTC decision
Contextual	Information adds meaning	Population, civic places, employment centres	Does not prove transit performance

These classifications travel with the data. They belong in APIs, database records, model outputs, interface labels and methodology. They should not be added as a disclaimer after a chart has already made a stronger visual claim.

The semantic layer supports this discipline. A route, stop, vehicle observation, access screen, journey leg and policy snapshot each have a defined meaning, grain, key, source, allowed joins and interpretation boundary. Shared definitions stop the same word from meaning one thing on a public page and another inside an experiment.

> **The practical rule**
>
> Observed, scheduled, estimated, modeled and contextual evidence can appear together, but they should not be silently blended into a score. Any combination must have a documented method and a reason that matters to the civic question.

04 / DATA ARCHITECTURE

Treat each output as a small, governed data product

A map, route profile, report card or agent result is not simply the final step in a query. Each is a data product with an audience, source contract, refresh expectation, quality threshold and retirement condition.

At the source layer, every connection should document ownership, licence, expected update frequency, timeouts, schema assumptions, minimum usable coverage and fallback behavior. The current platform uses TTC static and real-time feeds, City of Toronto open data, grouped Statistics Canada Census context, OpenStreetMap places and walking routes, Metrolinx open data, approved event listings and optional consented Microsoft Clarity exports.

At the normalization layer, source-specific structures become stable internal contracts. The transformation preserves source identity, source and retrieval timestamps, evidence class, quality state, missing fields, warnings and model version where applicable.

Storage pattern	Use	Architecture reason
Static JSON and geographic assets	Stable reference data and prepared public evidence	Low-cost delivery and reproducible builds
Bounded application cache	Repeated route, place or path evidence	Avoid duplicate external requests
Neon current state	Agent controls, active sessions, scheduler state	Consistent protected operations
Neon history	Observations, events, versions and experiment results	Review and reproducibility
Immutable snapshots	Dated route, ward or civic reports	Stable citation after publication

High-frequency collection requires a retention decision before it begins. Agent and similar evidence currently use a rolling 90-day default unless a longer period has a documented analytical purpose. The principle is plain: data should not be kept indefinitely merely because storage is available.

05 / TECHNOLOGY APPROACH

Give each provider a narrow, defensible job

A sustainable architecture does not ask every cloud service to do everything. It assigns responsibilities based on execution duration, state, delivery pattern and operational burden.

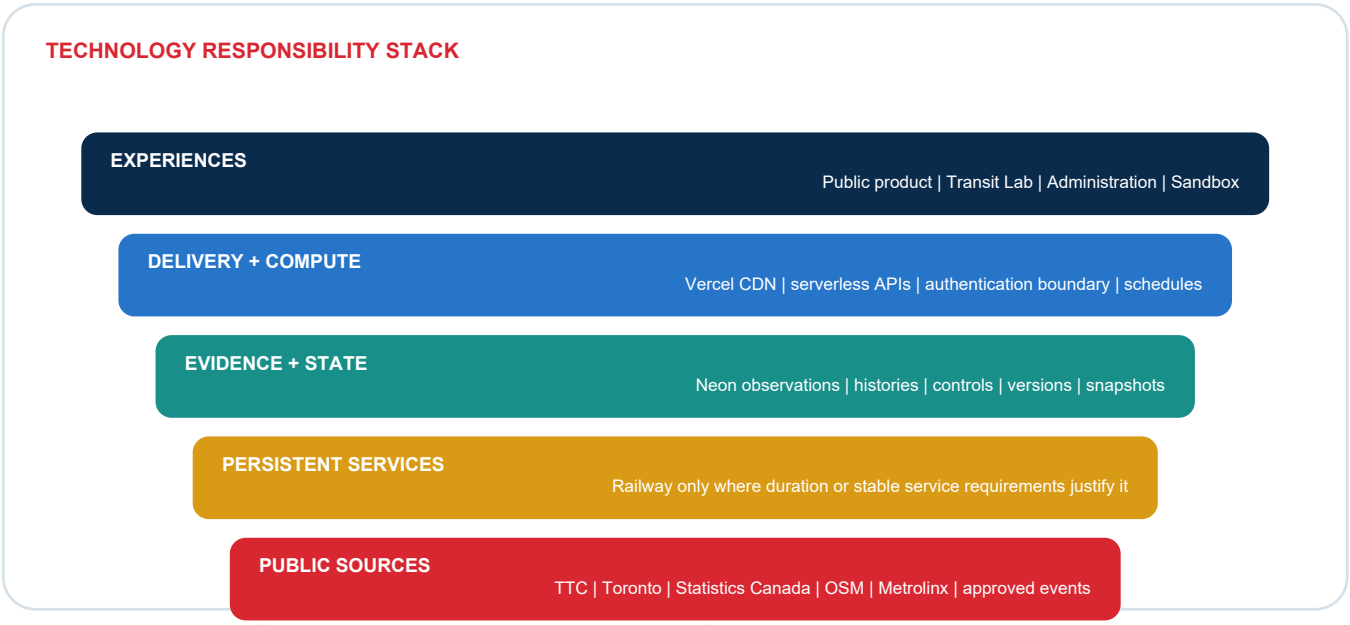


Figure 4. Technology choices follow workload characteristics rather than brand preference.

Layer	Responsibility	Boundary
Vercel	Static delivery, serverless APIs, authentication boundary, scheduled invocations, staged deployment	Avoid long-running or unbounded work
Neon Postgres	Structured history, agent state, model versions, controls and snapshots	Use restricted runtime access; reserve owner access for provisioning
Railway	Persistent or specialized services when serverless duration or state is unsuitable	Do not move workloads merely to imitate a traditional server
Browser + CDN	Maps, prepared geography, static reference evidence and interaction	Do not ship unnecessarily large source datasets

This arrangement is intentionally static-first. Stable pages, design assets, route shapes and prepared geographic files are delivered through the CDN. Functions are reserved for live, protected, scheduled or genuinely computational work. Historical and relational evidence goes to Neon. Persistent services are introduced only when their operating characteristics justify the additional cost and complexity.

06 / PROCESSING

Match computation to how quickly the evidence changes

Freshness is not one universal requirement. Vehicle positions, ward boundaries, event calendars and methodology pages change at different rates. Treating them alike creates unnecessary cost without improving the answer.

Pattern	Examples	Use when
Build time	Ward evidence, future-network models, multimodal graph, static bundle	Slower-changing or heavier preparation
Scheduled	Lab observations, gap evidence, agent checks, event calendar, baselines	Shared evidence with predictable refresh needs
Request time	Current vehicles, alerts, selected route or ward, protected interaction	Current selection or live source required
Persistent	Only workloads that cannot fit bounded serverless execution	Stable service or duration requirement

The recurring schedule is evidence-tiered: journey-agent checks can run every minute while enabled; route-gap observations every five minutes; broader civic observations every fifteen minutes; baselines and event calendars daily; route-place refresh and storage maintenance weekly. Those cadences are operating defaults, not permanent promises.

The decisive performance pattern is prepare once, reuse many times. The platform retrieves a source, validates it, calculates reusable evidence, stores or caches the result, and serves that result to many visitors. Public page traffic should not multiply city-scale geospatial calculations or upstream API calls.

> **A useful architecture can say no**
>
> Annual ridership forecasting was removed from the private Lab when its added civic value did not justify the ongoing workload. Retirement is part of architecture, not evidence of failure.

07 / JOURNEY AGENTS

Bounded autonomy, visible evidence and a complete passenger journey

The journey-agent experiment is intentionally not an unrestricted artificial intelligence system. It is a policy-bounded state machine that tests whether current and scheduled public evidence can support a representative trip from walking to waiting, boarding, riding, transferring, completing an activity and returning.

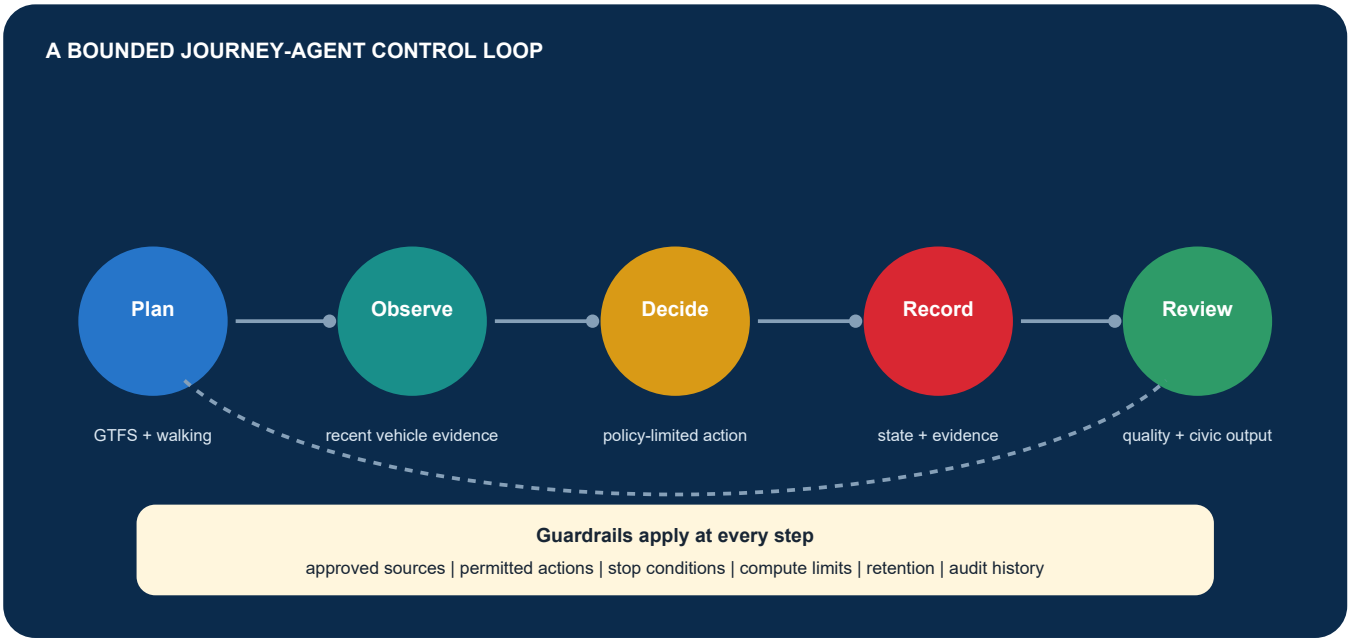


Figure 5. The agent's decisions are bounded by policy and supported by recorded evidence at each transition.

The architecture separates a planned transit leg from evidence that the journey progressed. A mapped stop proves that a stop exists. A scheduled trip proves intended service. A nearby vehicle report suggests that a bus may be approaching. None of those alone proves that a vehicle stopped and the agent boarded. The boarding transition therefore uses a defined stop-event rule and records the evidence used.

Each run carries a scenario, civic purpose, versioned policy, approved sources, planned path, state transitions, retry and stop conditions, and audit history. Multiple agents can share one source snapshot during a scheduler cycle rather than polling the same feed independently.

What the agent represents

It is a modelled representative journey, not a real rider, household or TTC operational record. Its value is the structured view of walking, waiting, riding and transfer evidence - and the places where that evidence fails to support a complete day.

08 / SECURITY AND PRIVACY

Protect the platform without hiding how evidence works

Transparency about methods does not require exposure of credentials, internal identifiers or protected operational controls. The architecture keeps those concerns separate.

+ Public APIs expose only information intended for publication; provider credentials remain server-side.
+ Transit Lab, administration and protected sandbox operations require authenticated sessions.
+ State-changing browser requests require authentication, an allowed method, a trusted origin and server-side validation.
+ Protected responses use private, no-store caching rules.
+ Content Security Policy, frame denial, MIME protection, restrictive permissions and secure cookies reduce browser attack surface.
+ Dependencies are audited, patched and regression-tested as part of release verification.

Privacy is designed around grouped evidence. The platform does not need resident-level tracking or household addresses to ask whether access differs across the city. Dissemination-area and ward context, privacy-screened reference points, suppression for small grouped populations and clearly modelled agents allow useful analysis without presenting individual behavior as observed fact.

Analytics is optional and consent-based. A new privacy review is required when a capability introduces finer geography, linked histories, user-submitted locations, communications or behavioral inference.

Transparency boundary

Public explainers should reveal purpose, data categories, method, safeguards and limitations. They should not reveal secrets, protected identifiers, detailed control paths or information that would weaken the security boundary.

09 / QUALITY AND RESILIENCE

A platform that knows when its evidence is weak

Public data is valuable, but it is not guaranteed to be complete, timely or stable. Reliability begins by assuming that an upstream feed will eventually be late, partial, malformed or unavailable.

Dimension	Operational question
Freshness	Is the evidence current enough for the claim?
Completeness	Are enough records and required fields present?
Validity	Do values follow expected formats and ranges?
Consistency	Do identifiers and measures retain meaning across time?
Coverage	Which routes, areas, periods or vehicles are missing?
Traceability	Can the result be tied to a source, method and version?

The platform uses request timeouts, route-level usability thresholds, bounded result sizes, uniqueness constraints, scheduler health records, last-valid-result fallbacks and explicit stale states. When one source fails, a page can preserve the evidence that remains valid while explaining what is missing.

Graceful degradation is not only a technical quality. It is part of civic honesty. The interface should tell people what remains known instead of replacing uncertainty with an empty map, a false zero or an overly confident sentence.

10 / GOVERNANCE

Move from experiment to publication through evidence gates

The private Transit Lab exists because development and publication are different acts. A model can run correctly and still be unsuitable for public interpretation.

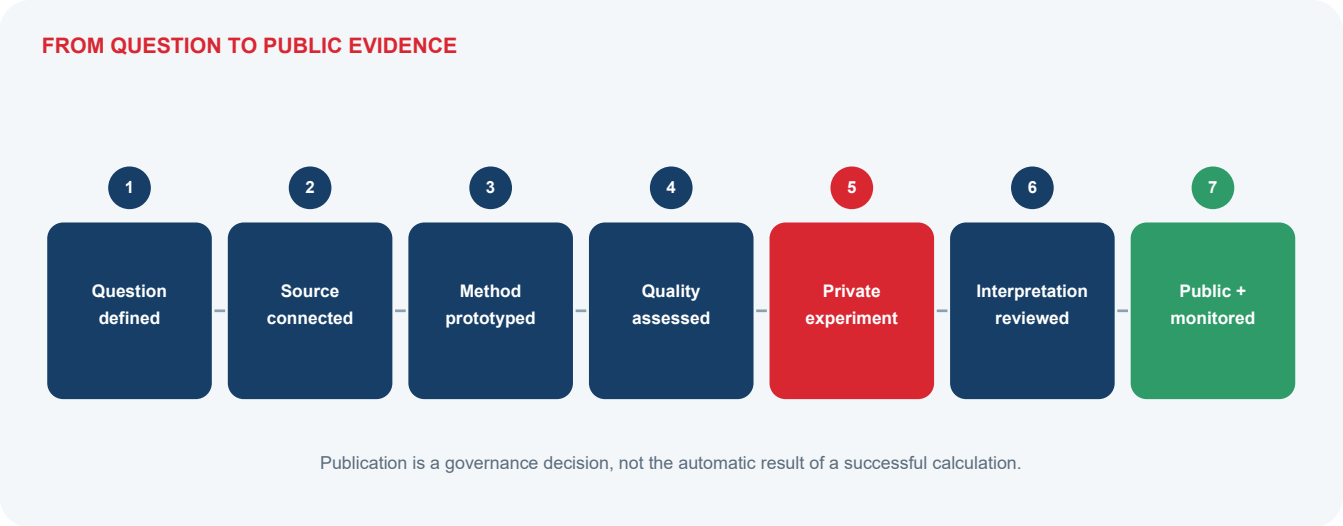


Figure 6. A successful calculation reaches the public only after quality, interpretation, privacy and operating reviews.

+ The civic question is understandable without internal terminology.
+ The method is stable, versioned and documented.
+ Sources are legally and operationally usable.
+ Quality and coverage support the intended claim.
+ Privacy controls match the geographic and behavioral detail.
+ Failure states are clear on desktop and mobile.
+ The output avoids unsupported causal or operational claims.
+ The cost and maintenance plan are sustainable.

The connected sandbox extends this logic into an enterprise-style workflow: register the use case, assign ownership, classify sources, define thresholds, approve actions and compute limits, run the controlled experiment, review exceptions, and approve, revise, pause or retire the capability.

11 / DELIVERY AND CHANGE

Staging is part of the architecture

A civic product should not rely on production as its first meaningful test. The release process treats the deployed artifact, its data references and its public interpretation as one package.

1	**Build**	Prepare evidence and generate the static bundle
2	**Verify**	Run integrity, security, domain and output checks
3	**Stage**	Deploy to a stable review environment
4	**Review**	Inspect evidence, wording, mobile behavior and accessibility
5	**Promote**	Move the exact staged artifact without rebuilding
6	**Canary**	Confirm production pages and runtime endpoints

Architecture decisions should be recorded when a change adds an infrastructure provider, persistent compute, a sensitive data class, a new evidence definition, materially different retention, an autonomous action, a public experiment or a cross-product dependency. The record should include context, alternatives, consequences, cost, privacy and rollback.

12 / APPLYING THE FRAMEWORK

Seven questions before adding the next feature

The framework is useful only if it changes day-to-day decisions. The following sequence converts a product idea into an architecture decision.

1 **Frame the civic question**	Write the question in language a resident could understand. Do not begin with a technology or dataset.
2 **Define the evidence contract**	Specify sources, evidence classes, grain, update need, quality threshold and interpretation boundary.
3 **Choose the maturity surface**	Place the capability in public, Transit Lab, administration or sandbox space.
4 **Select the lightest processing pattern**	Choose build-time, scheduled, request-time or persistent execution based on actual freshness and workload.
5 **Design failure and privacy behavior**	Decide what appears when data is missing and confirm the output cannot identify or misrepresent people.
6 **Instrument quality, cost and**	Record source time, coverage, version, provider use and operational health.
7 **Validate and publish deliberately**	Test wording, mobile layout, accessibility, security, civic interpretation and retirement conditions.

CONCLUSION

A civic platform earns trust by knowing its limits

The most important architectural decision in Toronto Public Transit Service Pulse is not the choice of Vercel, Neon, Railway, Leaflet or any individual data source. It is the decision to keep public meaning connected to evidence from beginning to end.

Static delivery, bounded serverless processing, structured history, semantic definitions, explicit evidence classes, protected experimentation and staged publication work as one system. Together they support complex questions without turning every experiment into a public claim.

The approach is deliberately conservative:

> **Collect only what supports a defined question.**
> **Compute only as often as the evidence requires.**
> **Publish only what can be responsibly explained.**
> **Preserve enough provenance to challenge or reproduce the result.**

That restraint allows the platform to expand - from live service and access screens to agent journeys, future networks and cross-product governance - without losing sight of its civic responsibility.

Architecture fitness checks

+ Can every public result be traced to a source and method?
+ Can a user distinguish observation, schedule, estimate, model and context?
+ Can a source fail without producing a misleading claim?
+ Can costs be attributed to providers and major workloads?
+ Can historical results retain the method and policy used at the time?
+ Can a feature be paused or retired when its value no longer justifies its burden?

Source note

Based on the platform's October 2026 architecture reference and implemented repository patterns. Toronto Public Transit Service Pulse is an independent civic product, not an official TTC platform or TTC website.